ASTR 427 HW 2: Comments

Akhila Anya Pisupati

October 26, 2026

1 AI Disclosure

1. I would ask it to explain my code to me line-by-line, just to make sure it is doing what I thought it was doing, and it would alert me to syntax errors I didn't realize were there, which was helpful.

2. I had a couple of traceback errors where it would tell me, say, my ylist was not defined even though I clearly defined it. The issue was where I was defining it, but I couldn't figure it out so I would only ask AI where the definition was supposed to go in the code, and I would move it around.

3. 3a: how to actually run the RK function. I ended up having to use "None" in the parameters for ylist and tlist to function properly

4. 3b: A large traceback error on defining the vector as 'x, y, vel_x, vel_y = vector' got me confused. I changed the line to x = vector[:,0], etc.

5. Numpy arrays took too long to run so I needed some help deciding which ones to convert to regular Python arrays. I also needed to write it as ylist = [np.array()] instead of np.array([]) to prevent errors.

6. I was very confused at the Euler error graph and needed to better understand how Euler method works, so I just wrote up my guess at why the error was like that and asked the AI to refine it a little bit.

2 Problem 2

2.1 Problem 2a

As the step size gets smaller, all the methods become better appproximations. You can see that for h = 1, none were approximations and RK/Leapfrog had spikes for what should be a standard cosine function, as we are given. This is because the big step size is missing the waves of the cosine function. In other words, the measurement done by both methods is occuring once every second which is only going to grab part of the cosine function itself (which of course repeats every 2π seconds.) Curves are really just infinite lines put together, so

the graph looks "spikey". But as the measurements become more frequent, they more closely resemble the curves of the cosine function because there is more integration being done in the same window.

For the Euler method, it is very unstable at high time steps because it assumes a constant slope for the entire time step. That means that for a whole 1 second it is a flat line, whereas the cosine function is so rapidly changing over that 1 second, and definitely does not resemble anything really linear in that large of a window. And when it calculates the next derivative after that 1 second, the actual point value is so offset from where that flat point left off that the correction has to become even more extreme, and so the Euler method does really bad with large time steps. This issue becomes much less notable when the flat lines are not as long, particularly when they are very small like 0.01 seconds, and so they more closely will resemble the cosine function.

2.2 Problem 2b

The errors do converge! We have to read this graph backwards because the stepsizes are getting smaller, and the log function increases (in the negative direction) for decreasing x. You can see the slope is about 4.6 for the RK method and about 2 for the Leapfrog method, which corresponds with an error on the order of h^2 for Leapfrog and h^4 for the RK, which we know from class. Meanwhile we know that Euler is a first-order calculation method to begin with, so the error is also first order and thus just a straight line with a slope of 1 at the end (I am a bit confused about the rest of the graph). Therefore this error expectedly converges!

3 Problem 3

3.1 Problem 3c

E(t) is much more dramatic for the RK method than for Leapfrog. I am not fully sure why, so here is my best guess: there is some sort of truncation error present in the RK method and the Hamiltonian provides greater accuracy for the Leapfrog method, as we discussed in class.

I didn't fully understand the explanation in class I must admit, but I do know Leapfrog is symplectic, meaning it describes a Hamiltonian . The Leapfrog method approximates this Hamiltonian then solves exactly, so it can provide a much better approximation for the energy. The RK method is not symplectic, on the other hand, so it has a truncation error problem.

The truncation error is using a fixed order with a Taylor series, and all the errors are just accumulating as a result. But the Leapfrog's Hamiltonian will more closely approximate the actual value because a Hamiltonian by its nature is related to some physical system which has constraints on what the values realistically can be. In this case energy has conservation rules, which makes sense since Hamiltonians govern conserved quantities, and there is no real

significance to, say, "infinite energy". RK is not bound by physical constraints and so can just keep going on forever.

It makes sense that the clock time for the RK method is larger because it has to calculate the derivative 4 separate times, which would require more memory usage, whereas the Leapfrog method has two separate derivative functions that are called separately outside of the function, and they really don't do much: One just returns the velocity, and the other returns the negative of the position (because this is a harmonic oscillator). Over 100k seconds those derivative calculations add up but Leapfrog bypasses them entirely and is able to thus run a good amount faster.